



Study on OpenMPI

Document No: SSDG/HPC-SoC/2022/ARM-MPI/Study/1.0
Version No.: 1.0
Control Status: Controlled (Internal Circulation Only)
Authors: Ashish Bisht, Deepika H V
Peer Reviewed By: Prachi Pandey, Shamjith K V
Approved By: Hari Babu P
Release By: HPC-SoC Group
Date of Release: 25/01/2023
Copy No.: 1

On-line Copy	Controlled Copy (Internal circulation only)	1.0
---------------------	--	------------



Table of Contents

1. INTRODUCTION	3
1.1 INTRODUCTION TO OPENMPI	3
1.2 PLUGIN ARCHITECTURE	4
1.3 FRAMEWORKS IN OPENMPI	5
2. NETWORK TOPOLOGY.....	7
2.1 NETWORK ROUTING ALGORITHMS.....	8
2.2 NETWORK INFERENCE.....	8
3. COMPARISON ON MPI CALLS IN ARM AND INTEL.....	9
3.1 PERFORMANCE COMPARISON OF OPENMPI AND MVAPICH2 ON ARM	10
3.2 MPI BASED POINT TO POINT COMMUNICATIONS	10
3.3 MPI BASED FILE IO	11
3.4 MPI COLLECTIVES	12
4. BEHAVIOURAL STUDY OF COLLECTIVE CALLS ON ARM	15
5. ONGOING EFFORTS ON OPENMPI ON ARM	17
6. INFERENCE ON OPENMPI PERFORMANCE.....	18
BIBLIOGRAPHY.....	19
APPENDIX A: OPENMPI COMPONENTS PATH.....	21
APPENDIX B: POINT TO POINT NON PERSISTENT MPI PROGRAM	25
APPENDIX C: POINT TO POINT PERSISTENT MPI PROGRAM	30
APPENDIX D: MPI BASED FILE IO PROGRAM-1	33
APPENDIX E: MPI BASED FILE IO PROGRAM-2.....	36
APPENDIX F: MATRIX MULTIPLICATION FOR ANALYSIS OF MPI COLLECTIVES.....	39
APPENDIX G: MPI_REDUCE PROGRAM	43
APPENDIX H: MPI_SEND, MPI_ISEND AND MPI_BSEND COMPARISON PROGRAM	47
APPENDIX I: MPI_RECV, MPI_Irecv COMPARISON PROGRAM	53



1. Introduction

Message Passing Interface (MPI) is a standardized and portable message passing standard designed to function on parallel computing architecture. The MPI standard defines the syntax and semantics of library routines that are useful for writing portable message passing programs in C, C++ and FORTRAN. MPI applications are highly parallel and compute intensive in nature.

There are several implementations of MPI available. This document explains the structure of one such implementation OpenMPI. It also introduces the network topology aspect of parallel computing and its effect on the message passing between two nodes in a parallel computing environment. In order to understand the difference between OpenMPI calls in two different architectures (Intel and ARM) we perform some experiments. The observations of these experiments have been documented further in this report.

The source of information / images obtained is stated wherever required.

1.1 Introduction to OpenMPI

OpenMPI is an open source MPI implementation of a standard library which is used for performing parallel processing on a distributed memory model. OpenMPI works on [Modular Component Architecture](#) (MCA) model. It is a series of frameworks, components, and modules that are [assembled at runtime](#) to create an MPI implementation. An MCA framework manages a specific Open MPI task for example creating a process, using a communication interface etc. OpenMPI groups similar functionalities together in distinct abstraction layers. These layers are:

- **Open Portable Access Layer (OPAL):** It provides utility such as generic linked list, string manipulation, debugging controls etc.
- **OpenMPI Runtime Time Environment (ORTE):** This layer provides support for parallel run time systems for example launching, monitoring and killing of processes. It also handles forwarding of stdin, stdout and stderr to respective processes.

- **Open MPI (OMPI):** All message passing semantics and MPI API are implemented on this layer.

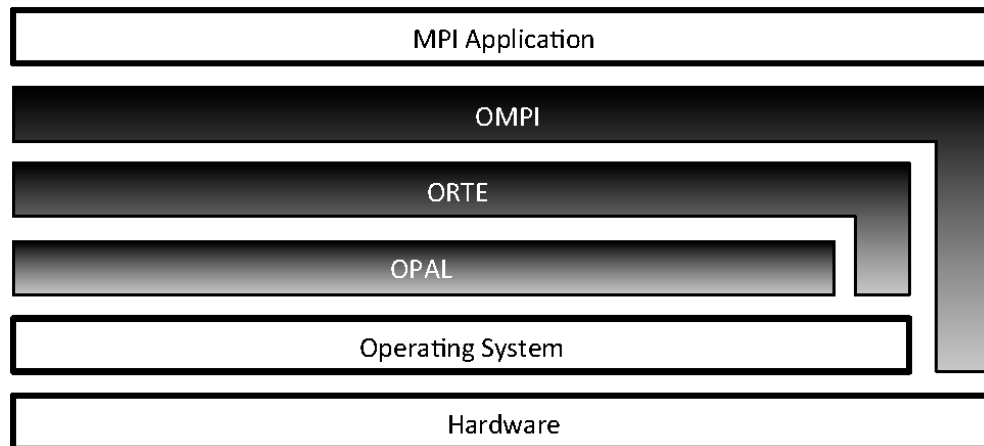


Fig-1: OpenMPI layers

(Source: <https://www.aosabook.org/images/openmpi/open-mpi-layers.png>)

1.2 Plugin Architecture

OpenMPI supports run time loadable components. These plugins can be configured using *mca* parameter during the execution of the application or can be set in advance in `<mpi_install_dir>/etc/openmpi_mca_params.conf` file and `<mpi_install_dir>/etc/pmix_mca_params.conf` file for all users. These files should be accessible to all other nodes running OpenMPI on the cluster. These configuration settings can be overridden by the user with *mca* parameter set during the execution of the application.

This provides user a way to load different components during the execution of the program. These components are grouped together depending on their framework. A framework can support only one kind of component at the runtime even though it might have multiple components. The figure below displays some of the important components.

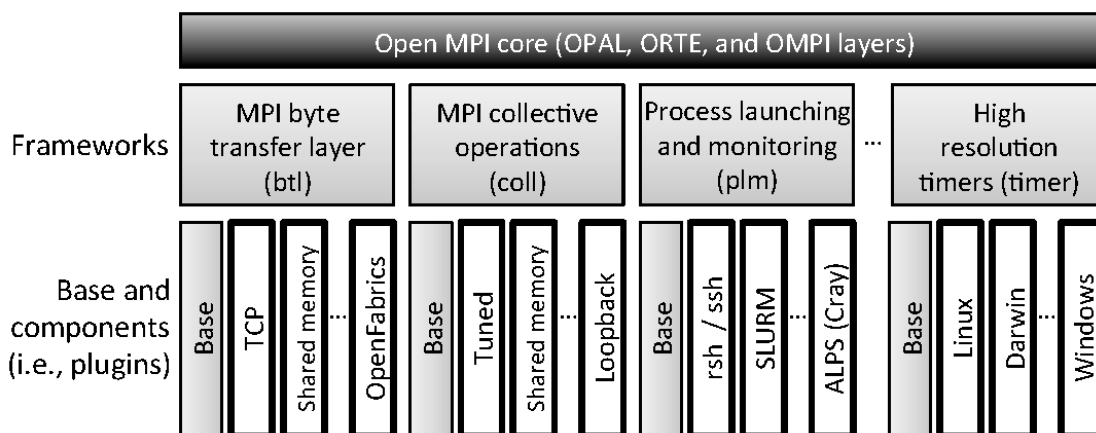


Fig-2: OpenMPI Frameworks/component Architecture

(Source:<https://www.aosabook.org/images/openmpi/open-mpi-mca.png>)

There are cases where neither a user nor the file specifies the component to be used for communication. In such a case where one has Ethernet and InfiniBand available, OpenMPI follows the so-called **Law of Least Astonishment**. Open MPI assumes that one will likely only want to use the high-speed network(s) for MPI message passing. Hence, the TCP BTL component will sense this and automatically deactivate itself.

That being said, **Open MPI may still use TCP for setup and tear-down information**, so one will see traffic across IP network during start-up and shutdown of the MPI job. This is normal and does not affect the MPI message passing channels.

1.3 Frameworks in OpenMPI

There are multiple frameworks defined in OpenMPI. Fig-3 lists these frameworks. These frameworks themselves are a collection of components. The relative path of these components in the OpenMPI directory is listed in *Appendix-A*.

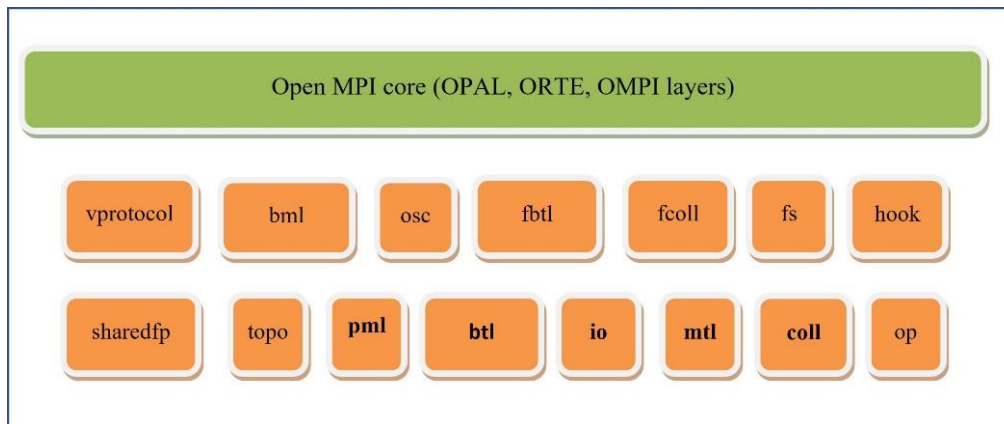


Fig-3: OpenMPI Frameworks

Note: **Bold text** in Fig-3 represents the frameworks mostly used by users. *Source of information:* https://www.openmpi.org/video/general/easybuild_tech_talks_01_OpenMPI_part2_20200708.pdf

Since OpenMPI operates in a distributive computer environment, it is very much dependent on the framework/component selected for execution of the program. Depending on the component selected for an execution, the performance may vary. Similarly the network topology aspect plays a vital role as a performance factor. The next section of the document deals with the network aspect of OpenMPI.

2. Network Topology

The choice of underlying network topology in a distributive computing environment is of great consequence. The approach of connecting different systems/nodes has a direct impact on the performance of communication between them. The top 500 supercomputers all rely on a fixed set of network topologies since they require the least number of delay/hops. These topologies are as follows:

- **Fat-tree:** In a fat tree topology the bandwidth at the higher levels of the tree structure is higher than at lower levels. Summit and Sierra are examples of supercomputers that use Fat-tree topology.
- **Dragonfly:** In this topology every router accommodates a set of terminal connections leading to end points and a set of topological connections leading to other routers. Frontier uses this network topology.
- **Hypercube:** In this topology $2n$ nodes are connected using an n -dimensional binary cube.
- **Torus:** Connecting the first and last node directly in a mesh topology results in torus. Torus is a highly scalable and can be made up n -dimensions. Cray Titan uses 3D torus, IBM BlueGene (BG/Q) used Torus 5D while Fugaku and PrimeHPC FX1000 uses 6D Torus implementations.

The following table shows the topology used with the number of nodes that are connected using the topology in the top supercomputers.

Network topology	Fat tree (Summit)	Torus 6D (Fugaku)	Dragonfly (Frontier)
Number of nodes	4608	158976	9408

Source: [https://en.wikipedia.org/wiki/Frontier_\(supercomputer\)](https://en.wikipedia.org/wiki/Frontier_(supercomputer)) , <https://www.olcf.ornl.gov/olcf-resources/compute-systems/summit/> , <https://www.fujitsu.com/global/about/innovation/fugaku/specifications/>

The least amount nodes connected by 6D Torus was 82,994 in Kei while Cray Titan connected 18,668 nodes using 3D Torus. Torus is generally used for **very large clusters** while Dragonfly and Fat-tree are used for **small/medium size clusters**.

2.1 Network Routing Algorithms

Routing algorithms define the path a packet would take on its transit from one node to another over the network. Therefore it becomes imperative that an appropriate routing algorithm for the given network topology is chosen. Selecting deterministic class of routing algorithms during certain [network traffic patterns](#) increases the overall communication time. Using Randomized class of algorithms can destroy [data locality](#), while using minimal path algorithms can end up causing [contention](#) in the network. Adaptive algorithms can create issues with the MPI programming as MPI needs messages to arrive at receiver process in a particular order but the adaptive algorithms can cause the [messages to arrive at any order](#). Knowing which class of algorithm to use in which scenario may help with increase in communication performance.

Algorithms like Depth First Search (DFS) and Breadth First Search (BFS) even though they work on graphs end up giving worse performance in a Torus topology. Algorithms like [XY Routing \[1\]](#), [Odd-even routing \[2\]](#), [Negative first turn models \[3\]](#), [Duato \[4\]](#), [*-channels \[5\]](#), [West first routing algorithm \[6\]](#), [Turn model \[6\]](#), [Dimension reversal \[7\]](#), [Dimension order routing \[8\]](#), [Balanced dimension order routing \[9\]](#) and [Extended balanced order routing \[10\]](#) all can be used with a network with torus topology.

Algorithms like [minus-first routing \[12\]](#), [AMLR \[13\]](#) and [dragonfly routing algorithm on multi-objects \[14\]](#) are some of the algorithms that can be used for routing in dragonfly network topology. For fat-tree network topology algorithms like: [weighted fat tree routing algorithm \[15\]](#) and routing algorithm proposed by [Zahavi \[16\]](#) are some of the choices available.

2.2 Network Inference

- Choosing right topology based on cluster size is imperative.
- Depending on the network topology selected the algorithm has to be chosen.

3. Comparison on MPI calls in ARM and Intel

There are multiple MPI implementations available. We choose two such open source implementations (OpenMPI and Mvapich2) for performance comparison. The result and conclusions drawn from the experiment is stated further in the document.

Almost all MPI based applications are majorly dependent on some core functionalities like sending/receiving data in a [point-to-point communication](#), [scattering/gathering](#) data to/from a large number of hosts and [reading/writing](#) a remote file. Simple programs that can demonstrate these functionalities were executed and profiled. The results of which have been recorded further down the document. The source code of the programs that were executed and analyzed is given in Appendix section of the report. GCC v12 (for OpenMPI) and ICC v19.1 (for IntelMPI) were used for the tests. The time recorded is in milliseconds stated otherwise.

Tools and Analysis Utilities (TAU) was used for profiling. TAU is a portable profiling and tracing toolkit for performance analysis of parallel programs written in FORTRAN, C++, C, Java and Python. We have specifically selected TAU as it can be used for both Intel and ARM platforms.

The specifications of systems used for the tests are stated below:

ARM: A64FX (Armv8.2-A SVE 512 bit architecture) @1.8GHz with 48 cores and 32 GB HBM2 memory. It has 38 such nodes in the cluster where each node is connected via HDR InfiniBand (Mellanox) - 200 Gbps. Network topology used is all to all. Filesystem: Lustre.

Intel: Intel(R) Xeon(R) Gold 8268 CPU @ 2.90GHz with 24 cores (total 48 cores/node as it is a dual socket system) and 192 GB DDR4 memory. It has 150 CPU nodes and 10 GPU nodes in the cluster where each node is connected via HDR InfiniBand (Mellanox) - 100Gbps. Network topology used is Fat-tree. Filesystem: Lustre.

Source: /proc/cpuinfo and /proc/meminfo files scontrol

3.1 Performance comparison of OpenMPI and Mvapich2 on ARM

A simple MPI based matrix multiplication program was executed using the open source MPI implementations - OpenMPI v4.1.1 and Mvapich2 v2.3.7 to compare the performance difference between them. The observations are presented below and source code for the program is given in *Appendix-F*. For the experiment GCC v12.1 was used as the base compiler for both MPI implementations. The readings presented below are an average of 3 runs of the program. The node configuration is 12N-4P. InfiniBand interface is used for experiments with Lustre filesystem. From the below table it can be observed that the OpenMPI v4.1.1 performs better among the two. [Thus, considering OpenMPI for all the experiments considered below.](#)

Table 1: performance comparison of open source MPI implementations on ARM

MPI implementation	Mvapich2 v2.3.7	OpenMPI v4.1.1
Time	29.5 mins	122 sec

3.2 MPI based Point to Point communications

A simple MPI application was profiled. In the program, a process sends n messages (4 byte each) using non persistent communications and persistent communications. The whole message exchange is just between two processes. The recorded data is presented below. The source code used for non-persistent communication and snapshot of the run is given in *Appendix-B*. The source code used for persistent communication is given in *Appendix-C*.

Case 1: Using non-persistent communication

No of communica tions	Intel				ARM	
	IntelMPI		OpenMPI			
	MPI_Send	MPI_Recv	MPI_Send	MPI_Recv	MPI_Send	MPI_Recv
1000	3	3	4	4	24	24
2000	7	7	8	8	48	48

Case 2: Using persistent communication with number of communications set to 1000

MPI Functions	Intel		ARM
	IntelMPI	OpenMPI	
MPI_Init	2250	787	152
MPI_Finalize	17	105	126
MPI_Start	5	4	23
MPI_Wait	9	4	22

Conclusion

- IntelMPI vs OpenMPI on Intel
 - *MPI_Send()*, *MPI_Recv()* and *MPI_Start()* has comparable performance on both MPI implementations.
 - *MPI_Init()* and *MPI_Wait()* have better performance in OpenMPI while *MPI_Finalize()* has better performance in IntelMPI.
- OpenMPI on Intel vs OpenMPI on ARM
 - OpenMPI on Intel is approximately 5-6x faster than ARM for *MPI_Send()*, *MPI_Recv()*, *MPI_Start()* and *MPI_Wait()* MPI functions.
 - Creation of process using *MPI_Init()* is better on ARM than Intel, though *MPI_Finalize()* takes more time on ARM.

3.3 MPI based File IO

A simple program was executed and profiled to test the file IO operations using MPI. The source code for the program along with snapshot can be found in *Appendix-D*.

MPI functions	Intel		ARM
	IntelMPI	OpenMPI	
MPI_Init	1413	321	158
MPI_Finalize	8	78	128
MPI_File_open	70	182	48
MPI_File_close	9	1	2

A sample program for testing *MPI_File_write_all()* was executed and profiled. Source code and snapshot of the run can be found in *Appendix-E*.

MPI functions	Intel		ARM
	IntelMPI	OpenMPI	
MPI_Init	809	713	163
MPI_Finalize	8	85	126
MPI_File_open	47	409	17
MPI_File_close	2	1	2
MPI_File_write_all	25	0.826	22
MPI_File_set_view	1	0.166	0.431

Conclusion

- IntelMPI vs OpenMPI on Intel
 - Closing a file has comparable performance for both MPI implementations
 - OpenMPI performs better when writing data to a shared file.
- OpenMPI on ARM vs OpenMPI on Intel
 - Opening a file shows better performance ($\sim 2.8x$ times) on ARM than using OpenMPI and IntelMPI on Intel platform.
 - Closing a file shows comparable performance to Intel.
 - Writing data to a shared file using OpenMPI on Intel has the best performance on Intel but the same doesn't apply for ARM. There is a degradation of performance by ~ 27 times.

3.4 MPI Collectives

A matrix multiplication program was executed across 4 nodes with 1 process/node and time was recorded for each major OpenMPI function calls. The source code for the program

along with the snapshot of the run is available in *Appendix F*. The time considered is inclusive time i.e. time spent inclusive of functions called by the function.

MPI functions	Intel		ARM
	IntelMPI	OpenMPI	
MPI_Init	943	768	164
MPI_Finalize	32	118	124
MPI_Scatter	450	642	2524
MPI_Gather	4775	4463	6028
MPI_Bcast	54	89	56

For *MPI_Reduce()*, Montecarlo PI calculation was executed and profiled using OpenMPI (on ARM & Intel) and IntelMPI (on Intel). Node configuration was set to 10N-4P. Observation data is presented below and source code along with snapshot of the run can be found in *Appendix-G*.

MPI Implementation	Intel		ARM
	IntelMPI	OpenMPI	
Time (msec)	54	18	3

Conclusion

- IntelMPI vs OpenMPI on Intel
 - IntelMPI performs slightly better than OpenMPI for scattering of data.
 - Broadcasting of data is better when using IntelMPI.
 - OpenMPI performs better for *MPI_Reduce()* and *MPI_Gather()* than IntelMPI.
- OpenMPI on ARM vs OpenMPI on Intel
 - Both scatter and gather of data have worse performance of about 4x and 1.3x times respectively when using OpenMPI on ARM. On further analysis it was found



that *MPI Collective Sync* are major cause for increase in the execution time for these MPI functions when compared to OpenMPI on Intel.

- Currently MPI_Bcast() and MPI_Reduce() shows better performance on ARM, though we suspect this is due to the current cluster topology of being connected all-to-all. This requires verification and study once the topology of the cluster is changed back to fat tree.

C-DAC Internal Use Only

4. Behavioural study of collective calls on ARM

From the observations in above section, it is clear that collective calls like scatter and gather suffer in ARM. In order to figure out the performance bottleneck of these functions we further analyzed these two functions. Both scatter and gather uses subroutines that are closely related to the *MPI_Send()* and *MPI_Recv()*. There exists different variants of send like: *MPI_Send()*, *MPI_Isend()* and *MPI_Bsend()*. The same is true for receive also. Therefore we compare performance of these variants using OpenMPI on ARM platform only. The observed data and conclusions drawn are stated further down the document.

Data recorded is an average value of 100 different runs with no other MPI activity over the network. Time recorded is in msec. Data size considered for transmission is 1 MB. The communication exchange is just between 2 processes. Source code can be found in the *Appendix* section of the report.

Case 1: Comparison of point-to-point calls of *MPI_Send()*, *MPI_Isend()* and *MPI_Bsend()*. Source code is stated in *Appendix-H*.

MPI functions	MPI_Send	MPI_Bsend + MPI_Buffer_attach + MPI_Buffer_detach	MPI_Isend + MPI_Wait
Time	7.85	1.02 + 0.02584 + 8.83	1.01 + 8.58

Case 2: Comparing *MPI_Recv()* and *MPI_Irecv()*. Source code is stated in *Appendix-I*.

MPI functions	MPI_Recv	MPI_Irecv + MPI_Wait
Time	11.56	0.04498 + 11.07

Conclusion

- On hindsight, *MPI_Send()* seems to perform the best among the variants but on further analysis we found that *MPI_Send()* was the most inconsistent among the variants. *MPI_Isend()* was much more consistent even with slightly higher average time.
- *MPI_Bsend()* has worst performance among all variants of *MPI_Send()* when considering the *MPI_Buffer_detach()*.
- *MPI_Irecv()* as a minute edge in performance when compared to *MPI_Recv()*. On further, analysis we find *MPI_Irecv()* performs better than its synchronous counterpart and the major time is consumed by *MPI_Wait()* which is approx. 95% of the time spent in the total point-to-point communication call.

5. Ongoing efforts on OpenMPI on ARM

The GitHub repository of OpenMPI does not have official support for ARM's SVE based instructions as on version 4.1.1.

Thus, we find independent efforts by some researchers, who have implemented SVE based instructions on reduction function for OpenMPI using ARM C Language Extension. In the paper [11], authors analyze ARM's SVE vector instruction set architecture and utilize those instructions to improve the *memcpy* function and various local reduction operations. The paper also proposes new strategies to improve the performance of various MPI functions like data packing/unpacking and MPI reduction. Using SVE based instructions for *memcpy* and reduction operations authors of the research paper claim to have observed **4X increase** in the performance of reduction operations. They further claim that by using SVE's gather and scatter based algorithm for packing and unpacking of data, the algorithm performs **2X faster** than the conventional algorithm being used.



6. Inference on OpenMPI Performance

It was observed that without optimizations performance for a program executed on ARM clearly suffers (for supporting data records check document titled: *ARM vs Intel Performance Report*). Hence it is imperative to supply optimization flags for the programs being executed on an ARM system.

Using only `-O3` optimization flag resulted in worse performance on ARM for MPI based programs. There are other GCC optimization flags that can be used in ARM which depend on the type of data being used by the program. These flags can also be found in the document titled: *Study of GCC behavior on A64FX (Document No: SSDG/HPC-SoC/2022/ARM/Study/1.0)*

From the above experiments, we can infer that all collective communication calls and point-to-point communication calls (`MPI_Send` and `MPI_Recv`) can be considered for ARM optimizations. We also find the possible improvements can be explored with asynchronous calls such as `MPI_Irecv()`. As it gives better performance provided we can optimize the `MPI_Wait()` function.

Along with the above suggested optimizations, we can further better the performance on ARM by using architecture specific optimization/techniques. Recorded data found in the documents titled: *ARM vs Intel Performance Report* and *Study of GCC behavior on A64FX* clearly shows that better performance can be achieved when using [SVE based instructions for a compute based MPI program](#). Research paper by D. Zhong et al [11] demonstrates improvement in the performance of OpenMPI reduction based functions when using SVE based instructions.

Since SVE helps with packing of data (for theory on SVE inner workings refer document titled: *Study of GCC behavior on A64FX*) and MPI applications are dependent on data packing and unpacking in MPI defined datatypes, we hereby infer that using SVE for data packing in OpenMPI would increase the performance of MPI based applications executed on A64FX.

Bibliography

1. W. Zhang, L. Hou, J. Wang, S. Geng and W. Wu, "Comparison Research between XY and Odd-Even Routing Algorithm of a 2-Dimension 3X3 Mesh Topology Network-on-Chip," 2009 WRI Global Congress on Intelligent Systems, 2009, pp. 329-333, doi: 10.1109/GCIS.2009.110.
2. Ge-Ming Chiu, "The odd-even turn model for adaptive routing," in IEEE Transactions on Parallel and Distributed Systems, vol. 11, no. 7, pp. 729-738, July 2000, doi: 10.1109/71.877831.
3. J. Chen, D. Xu and L. f. Xie, "A Fault-Tolerant Routing Protocol in 2D Torus Based on Positive-First and Negative-First Turn Models," 2009 International Conference on Information Engineering and Computer Science, 2009, pp. 1-5, doi: 10.1109/ICIECS.2009.5366017.
4. J. Duato, S. Yalamanchili, and L. Ni, "Interconnection Networks—An Engineering Approach," Morgan Kaufmann, 2002.
5. Berman, Pablo E., et al. "Adaptive deadlock-and livelock-free routing with all minimal paths in torus networks." *Proceedings of the fourth annual ACM symposium on Parallel algorithms and architectures*. 1992.
6. Glass, Christopher J., and Lionel M. Ni. "The turn model for adaptive routing." *ACM SIGARCH Computer Architecture News* 20.2 (1992): 278-287.
7. Y. Miura and S. Horiguchi, "An Adaptive Routing for Hierarchical Interconnection Network TESH," Proceedings of the 3rd International Conference on Parallel And Distributed Computing, Applications and Technologies, Kanazawa, 3-6 September 2002, pp. 335-342
8. M. A. Khan and A. Q. Ansari, "Quadrant-based XYZ dimension order routing algorithm for 3-D Asymmetric Torus Routing Chip (ATRC)," 2011 International Conference on Emerging Trends in Networks and Computer Communications (ETNCC), 2011, pp. 121-124, doi: 10.1109/ETNCC.2011.5958499.
9. Montañana, Jose Miguel, et al. "Balanced Dimension-Order Routing for k-ary n-cubes." *2009 International Conference on Parallel Processing Workshops*. IEEE, 2009.
10. Tyagi, S. H. I. V. A. M. "Extended balanced dimension ordered routing algorithm for 3D-networks." *International conference on parallel processing workshops*. 2009.
11. D. Zhong et al., "Using Arm Scalable Vector Extension to Optimize OPEN MPI," 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID), 2020, pp. 222-231, doi: 10.1109/CCGrid49817.2020.00-71.

Study on OpenMPI

12. D. Xiang, B. Li and Y. Fu, "Fault-Tolerant Adaptive Routing in Dragonfly Networks," in IEEE Transactions on Dependable and Secure Computing, vol. 16, no. 2, pp. 259-271, 1 March-April 2019, doi: 10.1109/TDSC.2017.2693372.
13. L. Zhu, H. Gu, X. Yu and W. Sun, "AMLR: An Adaptive Multi-Level Routing Algorithm for Dragonfly Network," in IEEE Communications Letters, vol. 25, no. 11, pp. 3533-3536, Nov. 2021, doi: 10.1109/LCOMM.2021.3104944.
14. Y. Qian, S. Pei, J. Yuan, D. Zhou, T. Liu and L. Kong, "DRAM: Dragonfly Routing Algorithm on Multi-objects by Optimal Thresholds," 2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCLOUD/SocialCom/SustainCom), 2021, pp. 135-142, doi: 10.1109/ISPA-BDCLOUD-SocialCom-SustainCom52081.2021.00031.
15. Y. Qian, S. Pei, J. Yuan, D. Zhou, T. Liu and L. Kong, "DRAM: Dragonfly Routing Algorithm on Multi-objects by Optimal Thresholds," 2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCLOUD/SocialCom/SustainCom), 2021, pp. 135-142, doi: 10.1109/ISPA-BDCLOUD-SocialCom-SustainCom52081.2021.00031.
16. Zahavi, Eitan, et al. "Optimized InfiniBand™ fat-tree routing for shift all-to-all communication patterns." *Concurrency and Computation: Practice and Experience* 22.2 (2010): 217-231.



Appendix A: OpenMPI components path

Full forms:

1. bml: BTL multiplexing layer
2. btl: Byte transport layer
3. coll: MPI collectives
4. fbtl: MPI file byte transfer layer
5. fcoll: MPI file collectives
6. fs: MPI file management
7. hook: Generic hooks
8. io: MPI IO
9. mtl: Matching transport layer
10. op: MPI reduction operations
11. osc: MPI one sided communications
12. pml: MPI point-to-point communications
13. sharedfp: MPI shared file pointer operations
14. topo: MPI topologies
15. vprotocol: Virtual protocol API interposition

Path Format: `./<layer>/mca/<framework>/<component>`

```
./contrib/build-mca-comps-outside-of-tree/btl_tcp2_component.c
./mpi/mca/bml/r2/bml_r2_component.c
./mpi/mca/coll/adapt/coll_adapt_component.c
./mpi/mca/coll/basic/coll_basic_component.c
./mpi/mca/coll/cuda/coll_cuda_component.c
./mpi/mca/coll/demo/coll_demo_component.c
./mpi/mca/coll/ftagreed/coll_ftagreed_component.c
./mpi/mca/coll/han/coll_han_component.c
./mpi/mca/coll/hcoll/coll_hcoll_component.c
./mpi/mca/coll/inter/coll_inter_component.c
./mpi/mca/coll/libnbc/coll_libnbc_component.c
```

Study on OpenMPI

```
./ompi/mca/coll/monitoring/coll_monitoring_component.c
./ompi/mca/coll/portals4/coll_portals4_component.c
./ompi/mca/coll/self/coll_self_component.c
./ompi/mca/coll/sm/coll_sm_component.c
./ompi/mca/coll/sync/coll_sync_component.c
./ompi/mca/coll/tuned/coll_tuned_component.c
./ompi/mca/coll/ucc/coll_ucc_component.c
./ompi/mca/fttl/ime/fttl_ime_component.c
./ompi/mca/fttl/posix/fttl_posix_component.c
./ompi/mca/fttl/pvfs2/fttl_pvfs2_component.c
./ompi/mca/fcoll/dynamic/fcoll_dynamic_component.c
./ompi/mca/fcoll/dynamic_gen2/fcoll_dynamic_gen2_component.c
./ompi/mca/fcoll/individual/fcoll_individual_component.c
./ompi/mca/fcoll/vulcan/fcoll_vulcan_component.c
./ompi/mca/fs/gpfs/fs_gpfs_component.c
./ompi/mca/fs/ime/fs_ime_component.c
./ompi/mca/fs/lustre/fs_lustre_component.c
./ompi/mca/fs/pvfs2/fs_pvfs2_component.c
./ompi/mca/fs/ufs/fs_ufs_component.c
./ompi/mca/hook/comm_method/hook_comm_method_component.c
./ompi/mca/hook/demo/hook_demo_component.c
./ompi/mca/io/ompio/io_ompio_component.c
./ompi/mca/io/romio341/src/io_romio341_component.c
./ompi/mca/mtl/ofi/mtl_ofi_component.c
./ompi/mca/mtl/portals4/mtl_portals4_component.c
./ompi/mca/mtl/psm2/mtl_psm2_component.c
./ompi/mca/op/avx/op_avx_component.c
./ompi/mca/op/example/op_example_component.c
./ompi/mca/osc/monitoring/osc_monitoring_component.c
./ompi/mca/osc/portals4/osc_portals4_component.c
./ompi/mca/osc/rdma/osc_rdma_component.c
./ompi/mca/osc/sm/osc_sm_component.c
./ompi/mca/osc/ucx/osc_ucx_component.c
./ompi/mca/part/persist/part_persist_component.c
./ompi/mca/pml/cm/pml_cm_component.c
./ompi/mca/pml/example/pml_example_component.c
```



Study on OpenMPI

```
./ompi/mca/pml/monitoring/pml_monitoring_component.c
./ompi/mca/pml/obl/pml_obl_component.c
./ompi/mca/pml/ucx/pml_ucx_component.c
./ompi/mca/pml/v/pml_v_component.c
./ompi/mca/sharedfp/individual/sharedfp_individual_component.c
./ompi/mca/sharedfp/lockedfile/sharedfp_lockedfile_component.c
./ompi/mca/sharedfp/sm/sharedfp_sm_component.c
./ompi/mca/topo/basic/topo_basic_component.c
./ompi/mca/topo/example/topo_example_component.c
./ompi/mca/topo/treematch/topo_treematch_component.c
./ompi/mca/vprotocol/example/vprotocol_example_component.c
./ompi/mca/vprotocol/pessimist/vprotocol_pessimist_component.c
./opal/mca/backtrace/base/backtrace_component.c
./opal/mca/backtrace/execinfo/backtrace_execinfo_component.c
./opal/mca/backtrace/none/backtrace_none_component.c
./opal/mca/backtrace/printstack/backtrace_printstack_component.c
./opal/mca/base/mca_base_component_compare.c
./opal/mca/btl/ofi/btl_ofi_component.c
./opal/mca/btl/portals4/btl_portals4_component.c
./opal/mca/btl/self/btl_self_component.c
./opal/mca/btl/sm/btl_sm_component.c
./opal/mca/btl/smcuda/btl_smcuda_component.c
./opal/mca/btl/tcp/btl_tcp_component.c
./opal/mca/btl/template/btl_template_component.c
./opal/mca/btl/uct/btl_uct_component.c
./opal/mca/btl/ugni/btl_ugni_component.c
./opal/mca/btl/usnic/btl_usnic_component.c
./opal/mca/dl/dlopen/dl_dlopen_component.c
./opal/mca/dl/libltdl/dl_libltdl_component.c
./opal/mca/memchecker/valgrind/memchecker_valgrind_component.c
./opal/mca/memory/malloc_solaris/memory_malloc_solaris_component.c
./opal/mca/memory/patcher/memory_patcher_component.c
./opal/mca/mpool/hugepage/mpool_hugepage_component.c
./opal/mca/mpool/memkind/mpool_memkind_component.c
./opal/mca/patcher/overwrite/patcher_overwrite_component.c
./opal/mca/rcache/gpusm/rcache_gpusm_component.c
```



Study on OpenMPI

```
./opal/mca/rcache/grdma/rcache_grdma_component.c
./opal/mca/rcache/rgpusm/rcache_rgpasm_component.c
./opal/mca/rcache/udreg/rcache_udreg_component.c
./opal/mca/reachable/netlink/reachable_netlink_component.c
./opal/mca/reachable/weighted/reachable_weighted_component.c
./opal/mca/shmem/mmap/shmem_mmap_component.c
./opal/mca/shmem/posix/shmem_posix_component.c
./opal/mca/shmem/sysv/shmem_sysv_component.c
./opal/mca/smsc/cma/smsc_cma_component.c
./opal/mca/smsc/knem/smsc_knem_component.c
./opal/mca/smsc/xpmem/smsc_xpmem_component.c
./opal/mca/threads/argobots/threads_argobots_component.c
./opal/mca/threads/pthreads/threads_pthreads_component.c
./opal/mca/threads/qthreads/threads_qthreads_component.c
./opal/mca/timer/altix/timer_altix_component.c
./opal/mca/timer/darwin/timer_darwin_component.c
./opal/mca/timer/linux/timer_linux_component.c
./opal/mca/timer/solaris/timer_solaris_component.c
./oshmem/mca/atomic/basic/atomic_basic_component.c
./oshmem/mca/atomic/ucx/atomic_ucx_component.c
./oshmem/mca/memheap/buddy/memheap_buddy_component.c
./oshmem/mca/memheap/ptmalloc/memheap_ptmalloc_component.c
./oshmem/mca/scoll/basic/scoll_basic_component.c
./oshmem/mca/scoll/mpi/scoll_mpi_component.c
./oshmem/mca/scoll/ucc/scoll_ucc_component.c
./oshmem/mca/spml/ucx/spml_ucx_component.c
./oshmem/mca/sshmem/mmap/sshmem_mmap_component.c
./oshmem/mca/sshmem/sysv/sshmem_sysv_component.c
./oshmem/mca/sshmem/ucx/sshmem_ucx_component.c
```




Appendix B: Point to point non persistent MPI program

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#include<mpi.h>

#define MCW MPI_COMM_WORLD
#define NOC 2000

void main() {
    MPI_Status mpi_status;
    int mpi_err, mpi_rank;
    float buffer, value=1.0f;
    int i;
    mpi_err = MPI_Init(NULL, NULL);
    mpi_err = MPI_Comm_rank(MCW, &mpi_rank);
    if(mpi_rank) {
        for(i=0;i<NOC;i++) {
            mpi_err = MPI_Send(&value, 1, MPI_FLOAT, 0, 123, MCW);
            value++;
        }
        printf(" PID:%d buffer:%f\n", mpi_rank, value);
    } else {
        for(i=0;i<NOC;i++)
            mpi_err = MPI_Recv(&buffer, 1, MPI_FLOAT, 1, 123, MCW, &mpi_status);
        printf(" PID:%d buffer:%f\n", mpi_rank, buffer);
    }

    mpi_err = MPI_Finalize();
} // end of main()..
```



Study on OpenMPI

OpenMPI (Intel) performance snapshot

Number of communications: 1000

FUNCTION SUMMARY (total):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	24	1,674	2	2	837319	.TAU application
98.6	60	1,650	2	2006	825214	main
84.4	1,413	1,413	2	0	706830	MPI_Init()
10.0	167	167	2	0	83680	MPI_Finalize()
0.3	4	4	1000	0	4	MPI_Send()
0.3	4	4	1000	0	4	MPI_Recv()
0.0	0.056	0.056	2	0	28	MPI_Comm_rank()

Number of communications: 2000

FUNCTION SUMMARY (total):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	21	1,715	2	2	857675	.TAU application
98.7	95	1,693	2	4006	846876	main
80.3	1,376	1,376	2	0	688407	MPI_Init()
12.0	205	205	2	0	102816	MPI_Finalize()
0.5	8	8	2000	0	4	MPI_Send()
0.5	7	7	2000	0	4	MPI_Recv()
0.0	0.052	0.052	2	0	26	MPI_Comm_rank()

IntelMPI performance snapshot

Number of communications: 1000

FUNCTION SUMMARY (total):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	667	4,532	2	2	2266119	.TAU application
85.3	291	3,864	2	2006	1932151	main
77.6	3,514	3,514	2	0	1757454	MPI_Init()
1.1	50	50	2	0	25336	MPI_Finalize()
0.1	3	3	1000	0	4	MPI_Send()
0.1	3	3	1000	0	4	MPI_Recv()
0.0	0.093	0.093	2	0	46	MPI_Comm_rank()

Study on OpenMPI

Number of communications: 2000

FUNCTION SUMMARY (total):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	952	4,104	2	2	2052168	.TAU application
76.8	247	3,151	2	4006	1575752	main
68.9	2,828	2,828	2	0	1414143	MPI_Init()
1.5	60	60	2	0	30432	MPI_Finalize()
0.2	7	7	2000	0	4	MPI_Send()
0.2	7	7	2000	0	4	MPI_Recv()
0.0	0.095	0.095	2	0	48	MPI_Comm_rank()

OpenMPI (ARM) performance snapshot

Number of communications: 1000

FUNCTION SUMMARY (total):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	73	715	2	2	357622	.TAU application
89.8	34	642	2	2006	321042	main
44.8	320	320	2	0	160338	MPI_Init()
33.2	237	237	2	0	118898	MPI_Finalize()
3.4	24	24	1000	0	24	MPI_Send()
3.4	24	24	1000	0	24	MPI_Recv()
0.0	0.14	0.14	2	0	70	MPI_Comm_rank()

Number of communications:2000

FUNCTION SUMMARY (total):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	99	785	2	2	392942	.TAU application
87.4	36	686	2	4006	343438	main
39.5	310	310	2	0	155368	MPI_Init()
30.9	243	243	2	0	121550	MPI_Finalize()
6.2	48	48	2000	0	24	MPI_Send()
6.2	48	48	2000	0	24	MPI_Recv()
0.0	0.14	0.14	2	0	70	MPI_Comm_rank()



C-DAC Internal Use Only



C-DAC Internal Use Only



Appendix C: Point to point persistent MPI program

```
#include<stdio.h>
#include<time.h>
#include<mpi.h>

#define MCW MPI_COMM_WORLD
#define MAS MPI_ANY_SOURCE
#define MAT MPI_ANY_TAG
#define MSI MPI_STATUS_IGNORE

void main() {
    MPI_Status mpi_status;
    MPI_Request mpi_request;
    int mpi_rank, mpi_err, mpi_size;
    float message=1.0f, buffer;
    int count=1000, id;
    time_t st, et;
    mpi_err = MPI_Init(NULL, NULL);
    mpi_err = MPI_Comm_size(MCW, &mpi_size);
    mpi_err = MPI_Comm_rank(MCW, &mpi_rank);

    st = time(NULL);
    if(mpi_rank) {
        mpi_err = MPI_Send_init(&message, 1, MPI_FLOAT, 0, 123, MCW, &mpi_request);
        while(count!=0) {
            mpi_err = MPI_Start(&mpi_request);
            mpi_err = MPI_Wait(&mpi_request, MSI);
            count--;
            message++;
        }
        et = time(NULL);
        printf(" PID:%d buffer:%f time:%ld\n", mpi_rank, message, et-st);
    } else {
        mpi_err = MPI_Recv_init(&buffer, 1, MPI_FLOAT, MAS, MAT, MCW, &mpi_request);
```

Study on OpenMPI

```

while(count) {
    mpi_err = MPI_Start(&mpi_request);
    mpi_err = MPI_Wait(&mpi_request, &mpi_status);
    id = mpi_status.MPI_SOURCE;
    count--;
}

et = time(NULL);
printf(" PID:%d buffer:%f time:%ld\n", mpi_rank, buffer, et-st);
}

mpi_err = MPI_Finalize();
// end of main()..
}

```

OpenMPI (Intel) performance snapshot

FUNCTION SUMMARY (mean):					
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	10	936	1	1	936437 .TAU application
98.9	23	925	1	2005	925728 main
84.1	787	787	1	0	787522 MPI_Init()
11.3	105	105	1	0	105918 MPI_Finalize()
0.5	4	4	1000	0	4 MPI_Start()
0.4	4	4	1000	0	4 MPI_Wait()
0.0	0.0265	0.0265	1	0	26 MPI_Comm_size()
0.0	0.017	0.017	1	0	17 MPI_Comm_rank()
0.0	0.013	0.013	0.5	0	26 MPI_Send_init()
0.0	0.011	0.011	0.5	0	22 MPI_Recv_init()

IntelMPI performance snapshot

FUNCTION SUMMARY (total):					
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	617	5,480	2	2	2740483 .TAU application
88.7	287	4,863	2	4010	2431754 main
82.1	4,501	4,501	2	0	2250852 MPI_Init()
0.6	35	35	2	0	17757 MPI_Finalize()
0.3	18	18	2000	0	9 MPI_Wait()
0.2	10	10	1	0	10430 MPI_Send_init()
0.2	10	10	2000	0	5 MPI_Start()
0.0	0.092	0.092	2	0	46 MPI_Comm_size()
0.0	0.044	0.044	2	0	22 MPI_Comm_rank()
0.0	0.024	0.024	1	0	24 MPI_Recv_init()

OpenMPI (ARM) performance snapshot

FUNCTION SUMMARY (mean):

%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	48	381	1	1	381392 .TAU application
87.2	8	332	1	2005	332622 main
40.1	152	152	1	0	152851 MPI_Init()
33.1	126	126	1	0	126052 MPI_Finalize()
6.0	23	23	1000	0	23 MPI_Start()
5.8	22	22	1000	0	22 MPI_Wait()
0.0	0.071	0.071	1	0	71 MPI_Comm_size()
0.0	0.052	0.052	1	0	52 MPI_Comm_rank()
0.0	0.0295	0.0295	0.5	0	59 MPI_Send_init()
0.0	0.028	0.028	0.5	0	56 MPI_Recv_init()



Appendix D: MPI based file IO program-1

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define LEN 10

#define CHECK_ERR(func) { \
    if (err != MPI_SUCCESS) { \
        int errorStringLen; \
        char errorString[MPI_MAX_ERROR_STRING]; \
        MPI_Error_string(err, errorString, &errorStringLen); \
        printf("Error at line %d: calling %s (%s)\n", __LINE__, #func, errorString); \
    } \
}

/*----< main() >-----*/
int main(int argc, char **argv)
{
    char *filename;
    int err, rank, nprocs, cmode, omode;
    MPI_File fh;
    MPI_Info info;

    MPI_Init(&argc, &argv);
    err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    CHECK_ERR(MPI_Comm_rank);
    err = MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    CHECK_ERR(MPI_Comm_size);

    filename = "fileopen";
    if (argc > 1) filename = argv[1];

    /* Users can set customized I/O hints in info object */
```



Study on OpenMPI

```
info = MPI_INFO_NULL; /* no user I/O hint */

/* set file open mode */
cmode = MPI_MODE_CREATE; /* to create a new file */
cmode |= MPI_MODE_WRONLY; /* with write-only permission */

/* collectively open a file, shared by all processes in MPI_COMM_WORLD */
err = MPI_File_open(MPI_COMM_WORLD, filename, cmode, info, &fh);
CHECK_ERR(MPI_File_open to write);

/* collectively close the file */
err = MPI_File_close(&fh);
CHECK_ERR(MPI_File_close);

/* set file open mode */
omode = MPI_MODE_RDONLY; /* with read-only permission */

/* collectively open a file, shared by all processes in MPI_COMM_WORLD */
err = MPI_File_open(MPI_COMM_WORLD, filename, omode, info, &fh);
CHECK_ERR(MPI_File_open to read);

/* collectively close the file */
err = MPI_File_close(&fh);
CHECK_ERR(MPI_File_close);

prog_exit:
    MPI_Finalize();
    return 0;
}
```

Study on OpenMPI

OpenMPI (Intel) performance snapshot

FUNCTION SUMMARY (mean):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call	
100.0	10	601	1	1	601583 .TAU application	
98.2	6	590	1	8	590819 main	
53.5	321	321	1	0	321864 MPI_Init()	
30.3	182	182	2	0	91048 MPI_File_open()	
13.1	78	78	1	0	78970 MPI_Finalize()	
0.2	1	1	2	0	627 MPI_File_close()	
0.0	0.023	0.023	1	0	23 MPI_Comm_rank()	
0.0	0.0205	0.0205	1	0	20 MPI_Comm_size()	

IntelMPI performance snapshot

FUNCTION SUMMARY (mean):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call	
100.0	475	2,117	1	1	2117561 .TAU application	
77.6	140	1,642	1	8	1642410 main	
66.7	1,413	1,413	1	0	1413260 MPI_Init()	
3.3	70	70	2	0	35236 MPI_File_open()	
0.5	9	9	2	0	4832 MPI_File_close()	
0.4	8	8	1	0	8450 MPI_Finalize()	
0.0	0.042	0.042	1	0	42 MPI_Comm_rank()	
0.0	0.021	0.021	1	0	21 MPI_Comm_size()	

OpenMPI (ARM) performance snapshot

FUNCTION SUMMARY (mean):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call	
100.0	46	399	1	1	399540 .TAU application	
88.5	15	353	1	8	353414 main	
39.7	158	158	1	0	158748 MPI_Init()	
32.2	128	128	1	0	128728 MPI_Finalize()	
12.1	48	48	2	0	24208 MPI_File_open()	
0.5	2	2	2	0	1081 MPI_File_close()	
0.0	0.07	0.07	1	0	70 MPI_Comm_rank()	
0.0	0.0505	0.0505	1	0	50 MPI_Comm_size()	



Appendix E: MPI based file IO program-2

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define CHECK_ERR(func) { \
    if (err != MPI_SUCCESS) { \
        int errorStringLen; \
        char errorString[MPI_MAX_ERROR_STRING]; \
        MPI_Error_string(err, errorString, &errorStringLen); \
        printf("Error at line %d: calling %s (%s)\n", __LINE__, #func, errorString); \
    } \
}

/*----< main() >-----*/
int main(int argc, char **argv) {
    char *filename;
    int i, err, cmode, rank, buf[10];
    MPI_Offset offset;
    MPI_File fh;
    MPI_Status status;

    MPI_Init(&argc, &argv);

    err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    CHECK_ERR(MPI_Comm_rank);

    filename = "filesetview";
    if (argc > 1) filename = argv[1];

    /* open a file (create if the file does not exist) */
    cmode = MPI_MODE_CREATE | MPI_MODE_RDWR;
    err = MPI_File_open(MPI_COMM_WORLD, filename, cmode, MPI_INFO_NULL, &fh);
    CHECK_ERR(MPI_File_open);
```



Study on OpenMPI

```
/* initialize write buffer contents */
for (i=0; i<10; i++) buf[i] = 100 * rank + i;

/* set file offset for this calling process */
offset = (MPI_Offset)rank * 10 * sizeof(int);
/* MPI_File_set_view() sets the file visible region to this process starts
 * at offset. Setting etype argument to MPI_INT means this file will be
 * accessed in the units of integer size. Setting filetype argument to
 * MPI_INT means a contiguous 4-byte region (assuming integer size of 4
 * bytes) is recursively applied to the file to form the visible region to
 * the calling process, starting from its "offset" set in the offset
 * argument. In this example, the "file view" of a process is the entire file
 * starting from its offset.
 */
err = MPI_File_set_view(fh, offset, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
CHECK_ERR(MPI_File_set_view);
/* Each process writes 3 integers to the file region visible to it.
 * Note the file pointer will advance 3x4 bytes after this call.
 */
err = MPI_File_write_all(fh, &buf[0], 3, MPI_INT, &status);
CHECK_ERR(MPI_File_set_view);

/* Each process continues to write next 7 integers to the file region
 * visible to it, starting from the file pointer updated from the previous
 * write call.
 */
err = MPI_File_write_all(fh, &buf[3], 7, MPI_INT, &status);
CHECK_ERR(MPI_File_set_view);
/* close the file collectively */
err = MPI_File_close(&fh);
CHECK_ERR(MPI_File_set_view);

MPI_Finalize();
return 0;
}
```

Study on OpenMPI

OpenMPI (Intel) performance snapshot

FUNCTION SUMMARY (mean):

%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	10	1,235	1	1	1235030 .TAU application
99.1	13	1,224	1	8	1224168 main
57.7	713	713	1	0	713096 MPI_Init()
33.1	409	409	1	0	409210 MPI_File_open()
7.0	85	85	1	0	85984 MPI_Finalize()
0.1	1	1	1	0	1334 MPI_File_close()
0.1	0.826	0.826	2	0	413 MPI_File_write_all()
0.0	0.166	0.166	1	0	166 MPI_File_set_view()
0.0	0.0235	0.0235	1	0	24 MPI_Comm_rank()

IntelMPI performance snapshot

FUNCTION SUMMARY (mean):

%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	353	1,294	1	1	1294797 .TAU application
72.7	47	941	1	8	941392 main
62.5	809	809	1	0	809351 MPI_Init()
3.7	47	47	1	0	47752 MPI_File_open()
1.9	25	25	2	0	12606 MPI_File_write_all()
0.6	8	8	1	0	8344 MPI_Finalize()
0.2	2	2	1	0	2230 MPI_File_close()
0.1	1	1	1	0	1346 MPI_File_set_view()
0.0	0.071	0.071	1	0	71 MPI_Comm_rank()

OpenMPI (ARM) performance snapshot

FUNCTION SUMMARY (mean):

%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	45	386	1	1	386244 .TAU application
88.1	6	340	1	8	340258 main
42.4	163	163	1	0	163593 MPI_Init()
32.8	126	126	1	0	126734 MPI_Finalize()
5.8	22	22	2	0	11212 MPI_File_write_all()
4.6	17	17	1	0	17951 MPI_File_open()
0.7	2	2	1	0	2640 MPI_File_close()
0.1	0.431	0.431	1	0	432 MPI_File_set_view()
0.0	0.07	0.07	1	0	70 MPI_Comm_rank()

Appendix F: Matrix multiplication for analysis of MPI collectives

```
#include<stdio.h>
#include<time.h>
#include<stdlib.h>
#include<mpi.h>

#define size 5000
#define MCW MPI_COMM_WORLD
#define MAS MPI_ANY_SOURCE

float m1[size][size], m2[size][size];
float result[size][size];

void create_matrix() {
    int i, j;
    srand(time(0));
    for(i=0;i<size;i++) {
        for(j=0;j<size;j++) {
            m1[i][j] = rand()%5;
            m2[i][j] = rand()%5;
            result[i][j] = 0;
        }
    }
} // end of create_matrix(..

void display_matrix() {
    int i, j;
    printf(" m1:\n");
    for(i=0;i<size;i++) {
        for(j=0;j<size;j++)
            printf(" %f", m1[i][j]);
        printf("\n");
    }
    printf(" m2:\n");
```

Study on OpenMPI

```
for(i=0;i<size;i++) {
    for(j=0;j<size;j++)
        printf(" %f", m2[i][j]);
    printf("\n");
}
// end of display_matrix()..

void multiply_matrix(int start, int end) {
    int i,j,k;
    for(i=start;i<end;i++) {
        for(k=0;k<size;k++) {
            result[i][k] = 0;
            for(j=0;j<size;j++)
                result[i][k] += m1[i][j]*m2[j][k];
        }
    }
}
// end of multiply_matrix()..

void show_result() {
    int i, j;
    printf(" Result:\n");
    for(i=0;i<size;i++) {
        for(j=0;j<size;j++)
            printf(" %f", result[i][j]);
        printf("\n");
    }
}
// end of show_result()..

void main() {
    int mpi_err, mpi_rank, mpi_size;
    time_t st, et;
    int start, end, epp;
    mpi_err = MPI_Init(NULL, NULL);
    mpi_err = MPI_Comm_size(MCW, &mpi_size);
    mpi_err = MPI_Comm_rank(MCW, &mpi_rank);
    epp = size*size/mpi_size;
```


Study on OpenMPI

```

start = mpi_rank * size/mpi_size;
end = start + size/mpi_size;
if(mpi_rank) {
    mpi_err = MPI_Scatter(NULL, epp, MPI_INT, &m1[start][0], epp, MPI_INT, 0, MCW);
    mpi_err = MPI_Bcast(&m2[0][0], size*size, MPI_INT, 0, MCW);
    multiply_matrix(start, end);
    mpi_err = MPI_Gather(&result[start][0], epp, MPI_INT, NULL, epp, MPI_INT, 0, MCW);
} else {
    create_matrix();
    st = time(NULL);
    mpi_err = MPI_Scatter(&m1[0][0], epp, MPI_INT, &m1[0][0], epp, MPI_INT, 0, MCW);
    mpi_err = MPI_Bcast(&m2[0][0], size*size, MPI_INT, 0, MCW);
    multiply_matrix(start, end);
    mpi_err = MPI_Gather(&result[start][0], epp, MPI_INT, &result[0][0], epp, MPI_INT, 0, MCW);
    et = time(NULL);
    printf(" Size:%d Time:%ld\n", size, et-st);
    //show_result();
}
mpi_err = MPI_Finalize();
} // end of main()..

```

OpenMPI (Intel) performance snapshot

FUNCTION SUMMARY (mean):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	13	2:14.399	1	1	134399316	.TAU application
100.0	40	2:14.385	1	8.25	134385997	main
95.3	2:08.056	2:08.056	1	0	128056186	multiply_matrix
3.8	5,078	5,078	3	0	1692826	MPI_Collective_Sync
3.3	10	4,463	1	1	4463752	MPI_Gather()
0.6	768	768	1	0	768760	MPI_Init()
0.5	23	642	1	1	642644	MPI_Scatter()
0.2	206	206	0.25	0	825014	create_matrix
0.1	118	118	1	0	118478	MPI_Finalize()
0.1	82	89	1	1	89808	MPI_Bcast()
0.0	0.031	0.031	1	0	31	MPI_Comm_size()
0.0	0.0235	0.0235	1	0	24	MPI_Comm_rank()

IntelMPI performance snapshot

FUNCTION SUMMARY (mean):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	415	2:57.571	1	1	177571546	.TAU application
99.8	94	2:57.155	1	8.25	177155549	main
96.1	2:50.670	2:50.670	1	0	170670526	multiply_matrix
2.9	5,129	5,129	3	0	1709836	MPI Collective Sync
2.7	51	4,775	1	1	4775024	MPI_Gather()
0.5	943	943	1	0	943410	MPI_Init()
0.3	45	450	1	1	450079	MPI_Scatter()
0.1	134	134	0.25	0	539913	create_matrix
0.0	53	54	1	1	54176	MPI_Bcast()
0.0	32	32	1	0	32317	MPI_Finalize()
0.0	0.0415	0.0415	1	0	42	MPI_Comm_size()
0.0	0.024	0.024	1	0	24	MPI_Comm_rank()

OpenMPI (ARM) performance snapshot

FUNCTION SUMMARY (mean):						
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive usec/call	Name
100.0	27	26:51.937	1	1	1611937210	.TAU application
100.0	10	26:51.909	1	8.25	1611909240	main
99.4	26:42.165	26:42.165	1	0	1602165223	multiply_matrix
0.5	8,516	8,516	3	0	2838735	MPI Collective Sync
0.4	21	6,028	1	1	6028718	MPI_Gather()
0.2	22	2,524	1	1	2524926	MPI_Scatter()
0.1	833	833	0.25	0	3335931	create_matrix
0.0	164	164	1	0	164057	MPI_Init()
0.0	124	124	1	0	124524	MPI_Finalize()
0.0	50	56	1	1	56850	MPI_Bcast()
0.0	0.0803	0.0803	1	0	80	MPI_Comm_size()
0.0	0.0535	0.0535	1	0	54	MPI_Comm_rank()



Appendix G: MPI_Reduce program

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <mpi.h>

#include <omp.h>

#define NUMINTERVALS 10000000

double func(double x)
{
    return (4.0 / (1.0 + x * x));
}

/* Main Program */

int main(int argc, char *argv[])
{
    int        NoIntervals, interval, ScatterSize, Noofthreads;
    int        MyRank, Numprocs, threadid, start, end;
    int        iproc, Root = 0, valid;
    int        Source, Source_tag;
    int        Destination, Destination_tag;
    double     PI25DT = 3.141592653589793238462643;
    double     mypi, pi, h, totalsum, x, partialsum, finalsum;
    MPI_Status  status;
```

Study on OpenMPI

```
double start_time, end_time;

/* ....MPI Initialisation.... */

MPI_Init(&argc, &argv);

MPI_Comm_size(MPI_COMM_WORLD, &Numprocs);

MPI_Comm_rank(MPI_COMM_WORLD, &MyRank);

if (MyRank == Root) {

    printf("Number of processes:%d\n Number of intervals:%ld", Numprocs, NUMINTERVALS);

    NoIntervals=NUMINTERVALS;

}

MPI_Bcast(&NoIntervals, 1, MPI_INT, Root, MPI_COMM_WORLD);

if (NoIntervals <= 0) {

    if (MyRank == Root)

        printf("Invalid Value For Number Of Intervals ..... \n");

    MPI_Finalize();

    exit(-1);

}

if (NoIntervals % Numprocs != 0) {

    if (MyRank == Root)

        printf("NoofIntervals Cannot Be Evenly Distributed \n");

    MPI_Finalize();

    exit(0);

}

if (MyRank == Root)
```



Study on OpenMPI

```
ScatterSize = NoIntervals / Numprocs;

MPI_Bcast(&ScatterSize, 1, MPI_INT, Root, MPI_COMM_WORLD);

h = 1.0 / (double) NoIntervals;

totalsum = 0.0;

finalsum = 0.0;

partialsum=0.0;

start=MyRank * ScatterSize + 1 ;

end=MyRank * ScatterSize + ScatterSize;

for (interval = start; interval <= end; interval=interval+1)

    {

        x = h * ((double) interval - 0.5);

        partialsum = partialsum + func(x);

    }

mypi = partialsum * h;

start_time = MPI_Wtime();

MPI_Reduce(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, Root, MPI_COMM_WORLD);

end_time = MPI_Wtime();

if (MyRank == Root) {

    printf("\nPi Is Approximately %.16f\n",pi);

    printf(" Time:%lf\n", end_time-start_time);

}

MPI_Finalize();

}
```



Study on OpenMPI

OpenMPI (Intel) performance snapshot

FUNCTION SUMMARY (mean):					
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	16	1,777	1	1	1777833 .TAU application
99.1	28	1,761	1	100008	1761825 main
73.5	1,306	1,306	1	0	1306945 MPI_Init()
11.6	206	206	100001	0	2 func [THROTTLED]
7.0	124	124	1	0	124395 MPI_Finalize()
4.9	86	86	3	0	28963 MPI_Collective Sync
4.3	6	76	2	2	38235 MPI_Bcast()
1.1	1	18	1	1	18792 MPI_Reduce()
0.0	0.0251	0.0251	1	0	25 MPI_Comm_rank()
0.0	0.0212	0.0212	1	0	21 MPI_Comm_size()

IntelMPI performance snapshot

FUNCTION SUMMARY (mean):					
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	13	2,685	1	1	2685943 .TAU application
99.5	40	2,672	1	100008	2672806 main
82.7	2,220	2,220	1	0	2220059 MPI_Init()
11.6	124	312	2	2	156011 MPI_Bcast()
8.0	215	215	3	0	71954 MPI_Collective Sync
2.0	26	54	1	1	54501 MPI_Reduce()
1.2	32	32	1	0	32449 MPI_Finalize()
0.5	13	13	100001	0	0 func [THROTTLED]
0.0	0.0009	0.0009	1	0	1 MPI_Comm_size()
0.0	0.000575	0.000575	1	0	1 MPI_Comm_rank()

OpenMPI (ARM) performance snapshot

FUNCTION SUMMARY (mean):					
%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	26	697	1	1	697906 .TAU application
96.1	192	670	1	100008	670931 main
25.9	180	180	1	0	180932 MPI_Init()
20.7	144	144	1	0	144232 MPI_Finalize()
13.3	92	92	100001	0	1 func [THROTTLED]
8.1	11	56	2	2	28343 MPI_Bcast()
6.8	47	47	3	0	15903 MPI_Collective Sync
0.5	0.963	3	1	1	3667 MPI_Reduce()
0.0	0.00143	0.00143	1	0	1 MPI_Comm_size()
0.0	0.00108	0.00108	1	0	1 MPI_Comm_rank()



Appendix H: MPI_Send, MPI_Isend and MPI_Bsend comparison program

Program for MPI_Send

```
#include<stdio.h>

#include<time.h>

#include<stdlib.h>

#include<mpi.h>

#define size 250000

#define MCW MPI_COMM_WORLD

int sbuffer[size], rbuffer[size];

void create_buffer() {

    int i;

    srand(time(NULL));

    for(i=0;i<size;i++)

        sbuffer[i] = rand()%1000;

} // end of create_buffer()..

void main() {

    MPI_Status *mpi_status;

    int mpi_err, mpi_size, mpi_rank;

    mpi_err = MPI_Init(NULL, NULL);
```

Study on OpenMPI

```
mpi_err = MPI_Comm_size(MCW, &mpi_size);

mpi_err = MPI_Comm_rank(MCW, &mpi_rank);

if(mpi_rank) {

    // Receive data

    mpi_err = MPI_Recv(&rbuffer[0], size, MPI_INT, 0, 123, MCW, mpi_status);

    printf("\n Data received by PID:%d from root", mpi_rank);

} else {

    // Send data

    create_buffer();

    mpi_err = MPI_Send(&sbuffer[0], size, MPI_INT, 1, 123, MCW);

    printf("\n Data sent by root to PID:1");

}

printf("\n --PID:%d", mpi_rank);

mpi_err = MPI_Barrier(MCW);

mpi_err = MPI_Finalize();

printf("\n");

} // end of main()..
```

Program for MPI_Isend

```
#include<stdio.h>

#include<time.h>

#include<stdlib.h>

#include<mpi.h>
```




Study on OpenMPI

```
#define size 250000

#define MCW MPI_COMM_WORLD

#define MAT MPI_ANY_TAG

#define MAS MPI_ANY_SOURCE

int sbuffer[size], rbuffer[size];

void create_buffer() {
    int i;

    srand(time(NULL));

    for(i=0;i<size;i++)
        sbuffer[i] = rand();
} // end of create_buffer()..

void main() {

    MPI_Status *mpi_status;

    MPI_Request mpi_request;

    int mpi_err, mpi_size, mpi_rank;

    mpi_err = MPI_Init(NULL, NULL);

    mpi_err = MPI_Comm_size(MCW, &mpi_size);

    mpi_err = MPI_Comm_rank(MCW, &mpi_rank);

    if(mpi_rank) {
```



Study on OpenMPI

```
// Receive data

mpi_err = MPI_Recv(&rbuffer[0], size, MPI_INT, MAS, MAT, MCW, mpi_status);

printf("\n Data received by PID:%d from root", mpi_rank);

} else {

    // Send data

    create_buffer();

    mpi_err = MPI_Isend(&sbuffer[0], size, MPI_INT, 1, 123, MCW, &mpi_request);

    mpi_err = MPI_Wait(&mpi_request, MPI_STATUS_IGNORE);

    printf("\n Data sent by root to PID:1");

}

printf("\n --PID:%d", mpi_rank);

mpi_err = MPI_Barrier(MCW);

mpi_err = MPI_Finalize();

printf("\n");

} // end of main()..
```

Program for MPI_Bsend

```
#include<stdio.h>

#include<time.h>

#include<stdlib.h>

#include<mpi.h>
```



Study on OpenMPI

```
#define size 250000

#define MCW MPI_COMM_WORLD

int sbuffer[size], rbuffer[size];

void create_buffer() {
    int i;
    srand(time(NULL));
    for(i=0;i<size;i++)
        sbuffer[i] = rand()%1000;
} // end of create_buffer()..

void main() {
    MPI_Status *mpi_status;
    int mpi_err, mpi_size, mpi_rank;

    int mpi_buffer_size = MPI_BSEND_OVERHEAD + sizeof(int)*size;
    int *mpi_buffer = (int*)malloc(mpi_buffer_size);

    mpi_err = MPI_Init(NULL, NULL);
    mpi_err = MPI_Comm_size(MCW, &mpi_size);
    mpi_err = MPI_Comm_rank(MCW, &mpi_rank);

    if(mpi_rank) {
        // Receive data
```

Study on OpenMPI

```
    mpi_err = MPI_Recv(&rbuffer[0], size, MPI_INT, 0, 123, MCW, mpi_status);

    printf("\n Data received by PID:%d from root", mpi_rank);

} else {

    // Send data

    create_buffer();

    mpi_err = MPI_Buffer_attach(mpi_buffer, mpi_buffer_size);

    mpi_err = MPI_Bsend(&sbuffer[0], size, MPI_INT, 1, 123, MCW);

    mpi_err = MPI_Buffer_detach(&mpi_buffer, &mpi_buffer_size);

    printf("\n Data sent by root to PID:1");

}

free(mpi_buffer);

printf("\n --PID:%d", mpi_rank);

mpi_err = MPI_Barrier(MCW);

mpi_err = MPI_Finalize();

printf("\n");

} // end of main()..
```



Appendix I: MPI_Recv, MPI_Irecv comparison program

Program for MPI_Recv

```
#include<stdio.h>

#include<time.h>

#include<stdlib.h>

#include<mpi.h>

#define size 250000

#define MCW MPI_COMM_WORLD

#define MAT MPI_ANY_TAG

int sbuffer[size], rbuffer[size];

void create_data() {

    srand(time(NULL));

    int i;

    for(i=0;i<size;i++)

        sbuffer[i] = rand()%1000;

} // end of create_data()..

void main() {

    MPI_Status mpi_status;

    int mpi_err, mpi_rank, mpi_size;

    mpi_err = MPI_Init(NULL, NULL);
```



Study on OpenMPI

```
mpi_err = MPI_Comm_rank(MCW, &mpi_rank);

mpi_err = MPI_Comm_size(MCW, &mpi_size);

if(mpi_rank) {

    // Recv data

    mpi_err = MPI_Recv(&rbuffer[0], size, MPI_INT, 0, MAT, MCW, &mpi_status);

    printf("\n Data received by PID:1 from root");

} else {

    // Send data

    mpi_err = MPI_Send(&sbuffer[0], size, MPI_INT, 1, 123, MCW);

    printf("\n Data sent by root");

}

printf("\n --PID:%d", mpi_rank);

mpi_err = MPI_Barrier(MCW);

mpi_err = MPI_Finalize();

printf("\n");

} // end of main()..
```

Program for MPI_Irecv

```
#include<stdio.h>

#include<time.h>

#include<stdlib.h>

#include<mpi.h>
```



Study on OpenMPI

```
#define size 250000

#define MCW MPI_COMM_WORLD

#define MAT MPI_ANY_TAG

int sbuffer[size], rbuffer[size];

void create_data() {

    srand(time(NULL));

    int i;

    for(i=0;i<size;i++)

        sbuffer[i] = rand()%1000;

} // end of create_data(..

void main() {

    MPI_Request mpi_request;

    int mpi_err, mpi_rank, mpi_size;

    mpi_err = MPI_Init(NULL, NULL);

    mpi_err = MPI_Comm_rank(MCW, &mpi_rank);

    mpi_err = MPI_Comm_size(MCW, &mpi_size);

    if(mpi_rank) {

        // Recv data

        mpi_err = MPI_Irecv(&rbuffer[0], size, MPI_INT, 0, MAT, MCW, &mpi_request);

        mpi_err = MPI_Wait(&mpi_request, MPI_STATUS_IGNORE);

        printf("\n Data received by PID:1 from root");
```

Study on OpenMPI

```
} else {  
    // Send data  
  
    mpi_err = MPI_Send(&sbuffer[0], size, MPI_INT, 1, 123, MCW);  
  
    printf("\n Data sent by root");  
  
}  
  
printf("\n --PID:%d", mpi_rank);  
  
mpi_err = MPI_Barrier(MCW);  
  
mpi_err = MPI_Finalize();  
  
printf("\n");  
}  
    // end of main()..
```